Lab 2 Post-Lab Report

ELEC 475

Sidarth Prabaharan – 20351244

Lily Stock - 20351147

This report compares Base LeNet-5, Modernized LeNet, a five-stage AlexNet adapted to 64 x 64 inputs, and the PyTorch Model Zoo AlexNet. The experiments study normalization, data augmentation, model capacity, and cross-dataset generalization. BatchNorm produced the strongest short-horizon normalization result, while augmentation reduced the generalization gap but did not exceed the unaugmented checkpoint in the 15-epoch schedule.

1 Architectural Specifications

For each convolution or pooling layer, the output width and height were computed using Wout = floor((W + 2P - K) / S) + 1. Receptive field calculations used rout = rin + (K - 1)jin and jout = jinS, starting with r = 1 and j = 1.

1.1 Base LeNet-5

Base LeNet-5 used Tanh activations and 2 x 2 average pooling. The spatial sequence was 3 x 64 x 64 -> 6 x 60 x 60 -> 6 x 30 x 30 -> 16 x 26 x 26 -> 16 x 13 x 13. The classifier therefore received 2,704 values and used 2704 -> 120 -> 84 -> 10. Its final receptive field was 16 x 16 with a jump 4.

Layer	Parameter calculation	Parameters
Conv1	(5 x 5 x 3 + 1) x 6	456
Conv2	(5 x 5 x 6 + 1) x 16	2,416
FC1	(2,704 + 1) x 120	324,600
FC2	(120 + 1) x 84	10,164
FC3	(84 + 1) x 10	850
Total		338,486

1.2 Modernized LeNet

Modernized LeNet replaced Tanh with ReLU and average pooling with overlapping 3 x 3 max pooling. Padding of two preserved the first convolution at 64 x 64. The spatial sequence was 3 x 64 x 64 -> 6 x 64 x 64 -> 6 x 31 x 31 -> 16 x 31 x 31 -> 16 x 15 x 15. The classifier input was 3,600 instead of 2,704, which explains the larger dense layer and total parameter count. Its final receptive field was 19 x 19 with jump 4.

Layer	Parameter calculation	Parameters
Conv1	(5 x 5 x 3 + 1) x 6	456
Conv2	(5 x 5 x 6 + 1) x 16	2,416
FC1	(3,600 + 1) x 120	432,120
FC2	(120 + 1) x 84	10,164
FC3	(84 + 1) x 10	850
Total		446,006

1.3 AlexNet-64

Stage	Operation	Output
Input	RGB image	3 x 64 x 64
1	Conv 5 x 5, s2, p2, 64, norm, ReLU	64 x 32 x 32
	MaxPool 3 x 3, s2	64 x 15 x 15
2	Conv 5 x 5, s1, p2, 192, norm - ReLU	192 x 15 x 15
	MaxPool 3 x 3, s2	192 x 7 x 7
3	Conv 3 x 3, p1, 384, norm, ReLU	384 x 7 x 7
4	Conv 3 x 3, p1, 384, norm, ReLU	384 x 7 x 7
5	Conv 3 x 3, p1, 256, norm, ReLU	256 x 7 x 7
	MaxPool 3 x 3, s2	256 x 3 x 3
Head	Dropout - 2304 -> 1024 -> 1024 -> classes	10 or 200

The final feature map contained 256 x 3 x 3 = 2,304 values. Dropout with p = 0.5 was used before both hidden dense layers. The normalization option was none, Local Response Normalization with size 5, alpha = 0.0001, beta = 0.75 and k = 2, or BatchNorm2d. The final receptive field after the third max-pooling layer was 97 x 97 with jump 16.

AlexNet-64 component	Parameters with BatchNorm
Five convolution layers	3,188,672
Five BatchNorm2d layers	2,560
Linear 2304 -> 1024	2,360,320
Linear 1024 -> 1024	1,049,600
Linear 1024 -> 10	10,250
Total with BatchNorm	6,611,402
Total without BatchNorm	6,608,842

1.4 PyTorch Model Zoo AlexNet

The torchvision AlexNet baseline retained the standard five-convolution feature extractor and adaptive 6 x 6 average pooling. For a 64 x 64 input, the convolutional feature extractor reaches 256 x 1 x 1 after its final max pool: adaptive pooling maps that output to 256 x 6 x 6 so the standard classifier still receives 9,216 values. The classifier is 9216 -> 4096 -> 4096 -> classes. The final pre-adaptive feature has a nominal receptive field of 195 x 195, larger than the input image.

Zoo AlexNet component	Parameters
Convolutional features	2,469,696
Linear 9216 -> 4096	37,752,832
Linear 4096 -> 4096	16,781,312
Linear 4096 -> 10	40,970
Total	57,044,810

1.5 Receptive field comparison

Model	Progression shown as receptive field / jump	Final
Base LeNet-5	Conv1 5/1: Pool1 6/2: Conv2 14/2: Pool2 16/4	16/4
Modernized LeNet	Conv1 5/1: Pool1 7/2: Conv2 15/2: Pool2 19/4	19/4
AlexNet-64	Conv1 5/2: Pool1 9/4: Conv2 25/4: Pool2 33/8: Conv3 49/8: Conv4 65/8: Conv5 81/8: Pool3 97/16	97/16
Zoo AlexNet	Conv1 11/4: Pool1 19/8: Conv2 51/8: Pool2 67/16: Conv3 99/16: Conv4 131/16: Conv5 163/16: Pool3 195/32	195/32

2 Training and Evaluation Setup

All images were resized to 64 x 64. Cross-entropy loss and Adam were used with learning rate 0.001, weight decay 0.0001, batch size 128, and seed 42 for the primary experiments. Early stopping was disabled so the requested epoch schedules could be compared directly. Training ran on an NVIDIA Tesla T4. AlexNet normalization runs used 10 epochs, augmentation runs used 15 epochs, and the Tiny ImageNet challenge used 10 epochs.

Setting	Value
Input	3 x 64 x 64
Loss	Cross entropy
Optimizer	Adam
Learning rate	0.001
Weight decay	0.0001
Batch size	128
Primary seed	42
Additional seeds	123 and 2026
Device	NVIDIA Tesla T4 CUDA

2.1 Data transforms

The augmented CIFAR-10 pipeline used RandomCrop(64, padding=4, padding_mode='reflect'), RandomHorizontalFlip(p=0.5), and ColorJitter with brightness, contrast, and saturation set to 0.2. CIFAR-10 normalization used mean [0.4914, 0.4822, 0.4465] and standard deviation [0.2470, 0.2435, 0.2616]. Tiny ImageNet used mean [0.4802, 0.4481, 0.3975] and standard deviation [0.2764, 0.2689, 0.2821]. Validation data were resized, converted to tensors, and normalized without stochastic augmentation.

3 Empirical Results and Figures

3.1 Table 7.3a Architectural Configuration and Computational Complexity

Model	Conv depth	Normalization and regularization	Augmentation	Params M	FLOPs M	Mean epoch s	Latency ms
Base LeNet-5	2	None: Tanh: average pool	None	0.338	7.17	21.98	0.39
Modernized LeNet	2	ReLU: overlapping max pool	None	0.446	9.19	22.33	0.66
AlexNet No Norm	5	Dropout p=0.5	None	6.609	436.70	23.72	2.12
AlexNet LRN	5	LRN: dropout p=0.5	None	6.609	436.70	25.94	1.94
AlexNet BatchNorm	5	BatchNorm2d: dropout p=0.5	None	6.611	437.97	24.39	1.39
AlexNet BN and Aug	5	BatchNorm2d: dropout p=0.5	Crop: flip: jitter	6.611	437.97	52.56	1.39
PyTorch Zoo AlexNet	5	Zoo model: dropout	None	57.045	188.20	24.93	2.15

The profiler counted one multiply-accumulate as approximately two FLOPs. This convention makes the measured FLOP values higher than some nominal assignment estimates that count a multiply-accumulate as one operation. Parameter orders match the expected scales. Wall-clock time and latency are hardware- and run-dependent.

3.2 Table 7.3b Predictive Performance Across Datasets

Model	CIFAR-10 val accuracy	CIFAR-10 macro F1	Tiny ImageNet val accuracy	Tiny ImageNet macro F1
Base LeNet-5	54.69%	54.28%	11.78%	9.72%
Modernized LeNet	66.36%	66.11%	20.01%	17.94%
AlexNet No Norm	72.83%	72.78%	16.66%	14.15%
AlexNet LRN	73.46%	73.46%	0.50%	0.00%
AlexNet BatchNorm	78.23%	78.20%	30.63%	29.11%

Model	CIFAR-10 val accuracy	CIFAR-10 macro F1	Tiny ImageNet val accuracy	Tiny ImageNet macro F1
AlexNet BN and Aug	81.26%	81.17%	28.91%	26.94%
PyTorch Zoo AlexNet	66.74%	66.72%	0.53%	0.01%

3.3 Extended Ablation Exploration

Comparison	Measured result	Delta and interpretation
Modernized vs Base LeNet	66.36% vs 54.69%	+11.67 accuracy points
LRN vs no normalization	73.46% vs 72.83%	+0.63 points: small gain
BatchNorm vs no normalization	78.23% vs 72.83%	+5.40 points: largest norm gain
Augmented vs unaugmented best	81.26% vs 82.42%	-1.16 points at this horizon
Augmentation generalization gap	4.65 vs 8.20 points	-3.56 points after rounding
Manual BatchNorm vs Zoo AlexNet	78.23% vs 66.74%	+11.49 points with fewer parameters

3.4 Multi-seed BatchNorm evaluation

Seed	Validation accuracy	Macro F1
42	78.23%	78.20%
123	79.08%	78.70%
2026	80.42%	80.48%
Mean +/- sample standard deviation	79.24 +/- 1.10%	79.13 +/- 1.20%

The spread across three random seeds was modest relative to the 5.40-point advantage of BatchNorm over the unnormalized model, so the normalization conclusion was not caused by one unusually favorable initialization.

3.5 Ablation summary

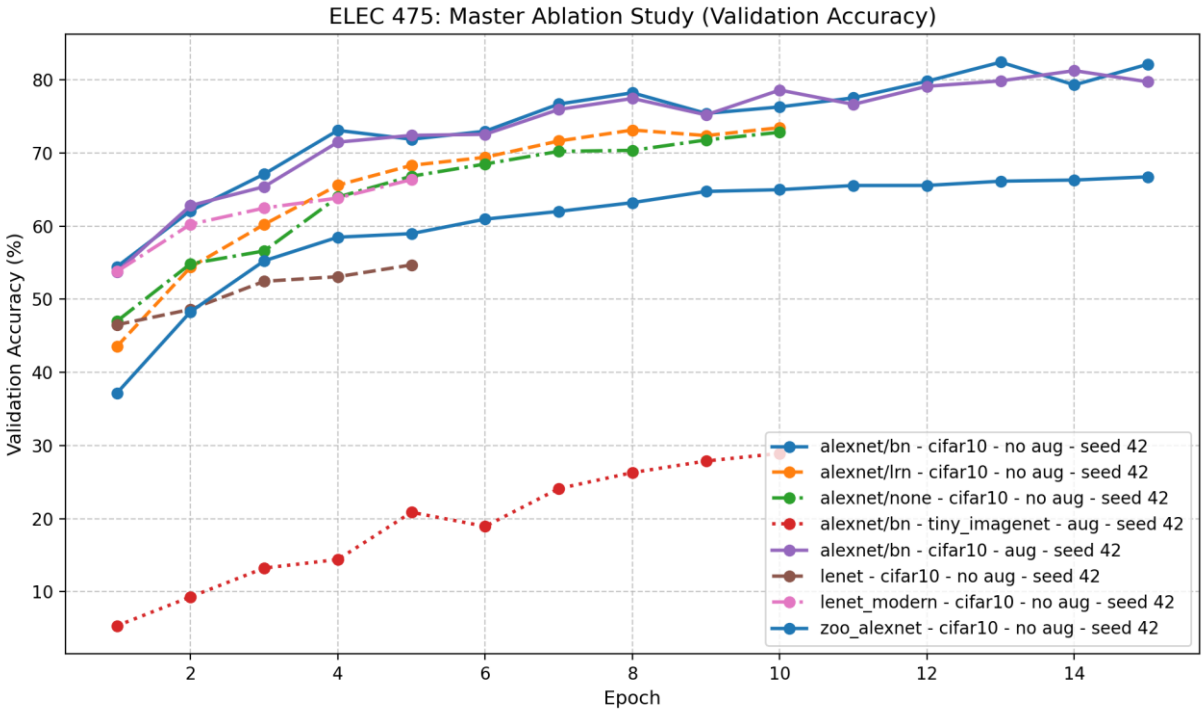


Figure 1. Comparative validation accuracy and macro F1 for the primary CIFAR-10 experiments.

The model ladder improved from Base LeNet to Modernized LeNet and then to the AlexNet variants. BatchNorm was the strongest normalization choice. The augmented AlexNet produced the best reported CIFAR-10 result in Table 7.3b, while the unaugmented 15-epoch run had a slightly higher peak during the focused augmentation comparison.

3.6 Normalization training curves

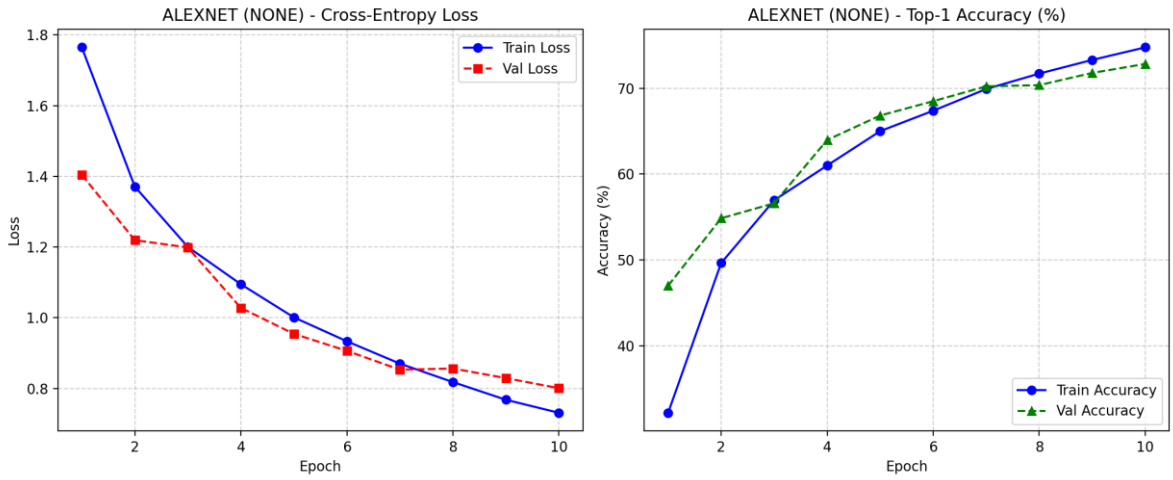


Figure 2. AlexNet without normalization on CIFAR-10.

Without normalization, the final training and validation accuracies were 74.76% and 72.83%. Validation loss decreased to 0.8007 by epoch 10 with one temporary rise. The trajectory was comparatively smooth, but convergence was slower and the final accuracy was lower than BatchNorm.

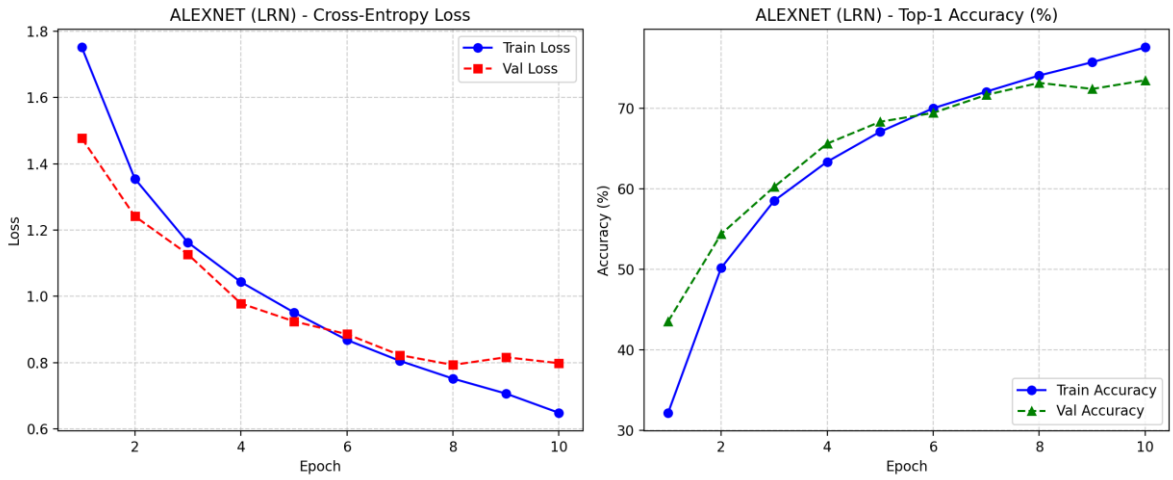


Figure 3. AlexNet with Local Response Normalization on CIFAR-10.

LRN finished at 77.54% training accuracy and 73.46% validation accuracy. Its minimum validation loss was 0.7933 at epoch 8 and the final loss was 0.7984. The 0.63-point accuracy gain over no normalization was small compared with its extra epoch time.

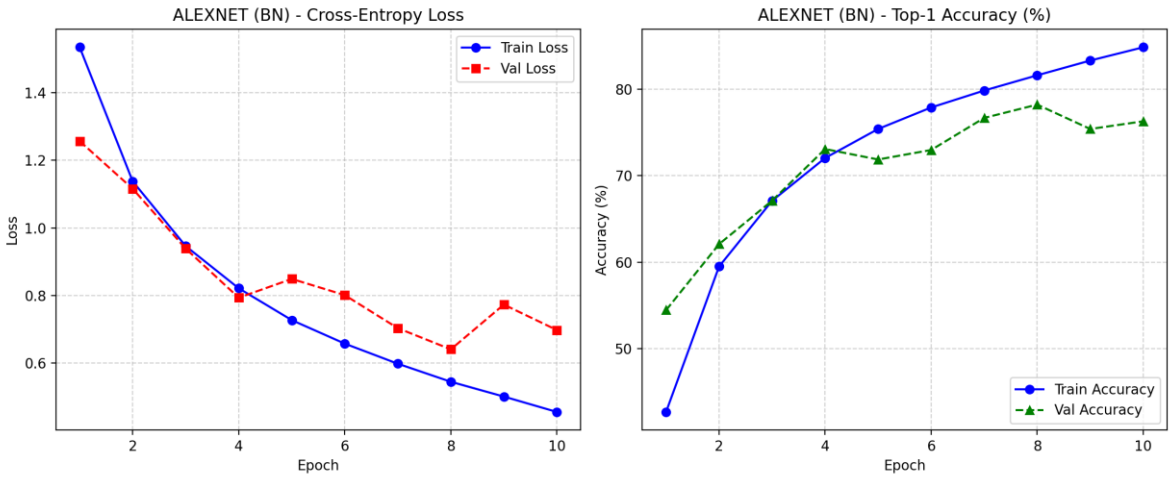


Figure 4. AlexNet with Batch Normalization on CIFAR-10.

BatchNorm reached its best evaluation at epoch 8 with 78.23% validation accuracy and 78.20% macro F1. Its minimum validation loss was 0.6404 at epoch 8 and the final value was 0.6976. The curve reached useful accuracy faster, although its validation loss had more short-horizon oscillation than the other two variants. Therefore, the observed benefit was faster and better optimization, not a perfectly monotonic curve.

3.7 Data augmentation curves and gap

Configuration	Final train accuracy	Final val accuracy	Gap	Best val accuracy	Best macro F1	Mean epoch
No augmentation	90.34%	82.14%	8.20	82.42%	82.44%	24.65 s
Crop, flip and jitter	84.38%	79.73%	4.65	81.26%	81.17%	52.56 s

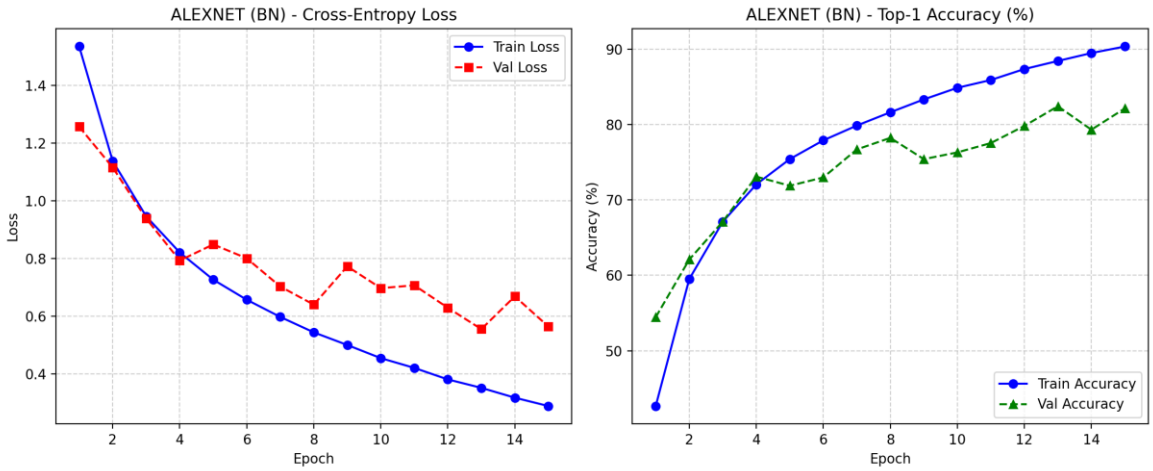


Figure 5. AlexNet-BatchNorm without augmentation for 15 epochs.

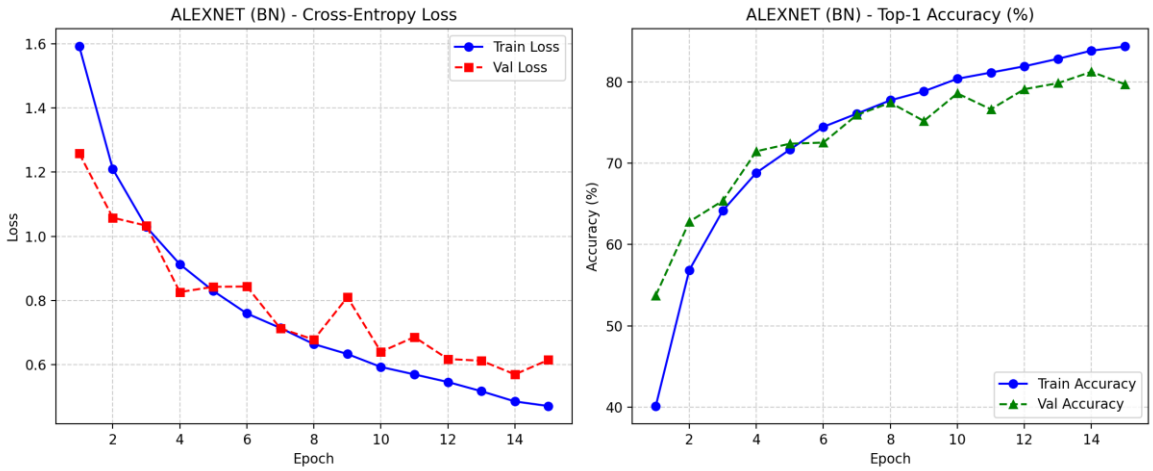


Figure 6. AlexNet-BatchNorm with crop, flip and color jitter for 15 epochs.

Augmentation reduced the final generalization gap by approximately 3.56 percentage points, from 8.20 to 4.65. It also made each epoch substantially slower because transforms were sampled online. The augmented model did not exceed the unaugmented validation metrics within 15 epochs, which is consistent with the augmented distribution being harder to fit rather than evidence that the transforms were invalid.

3.8 Tiny ImageNet benchmark

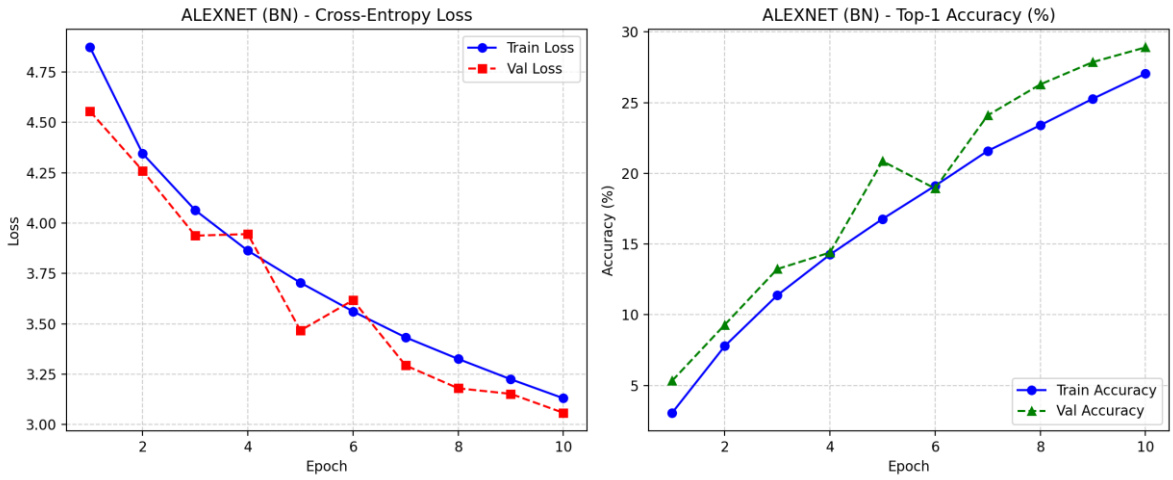


Figure 7. AlexNet-BatchNorm with augmentation on Tiny ImageNet-200.

The required Tiny ImageNet run reached 28.91% validation accuracy and 26.94% macro F1. Random guessing among 200 classes is 0.50%, so the observed accuracy was about 57.8 times chance. Validation loss ended at 3.0578 after two temporary rises and was still improving at epoch 10. The run did not show an early permanent plateau or divergence.

3.9 Exploratory data analysis

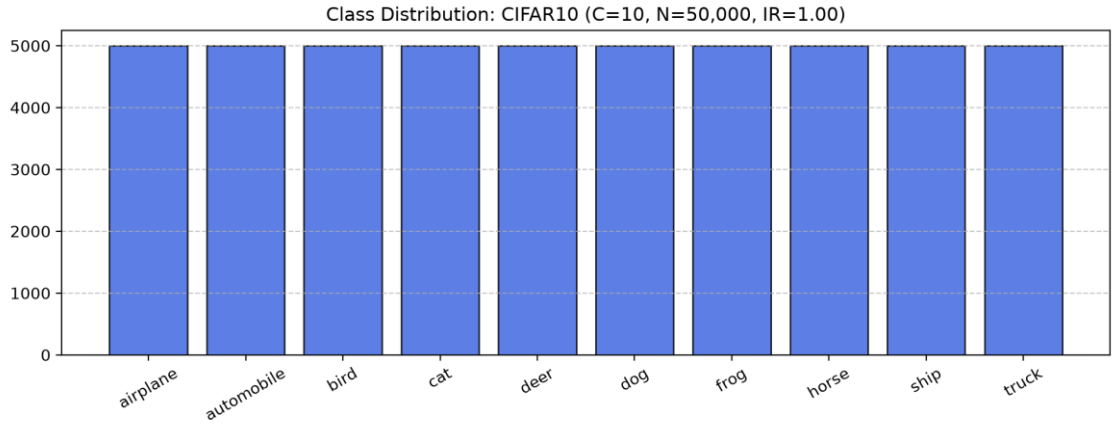


Figure 8. CIFAR-10 class distribution.

CIFAR-10 is balanced across its ten classes, so macro F1 and accuracy are expected to remain close unless particular classes are much harder than others.

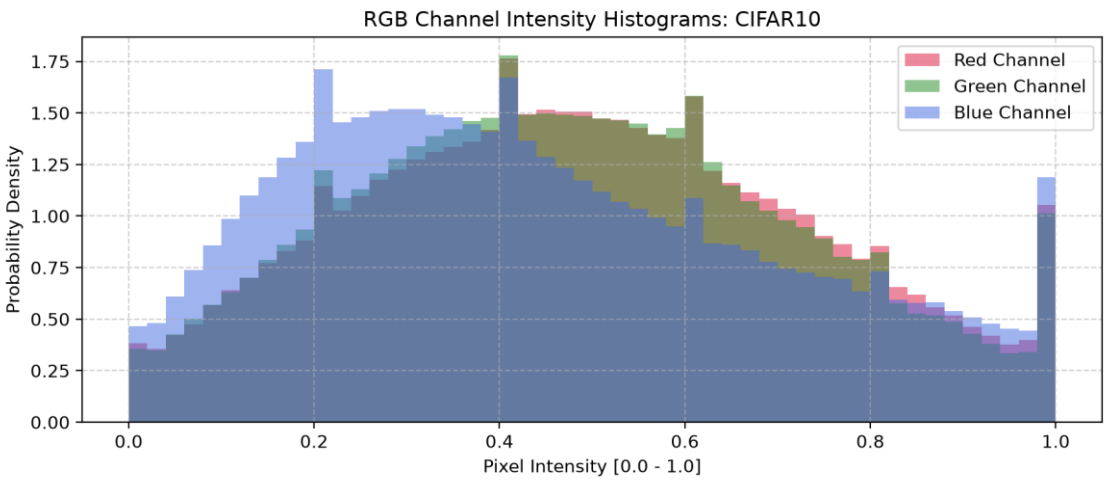


Figure 9. CIFAR-10 channel intensity distributions.

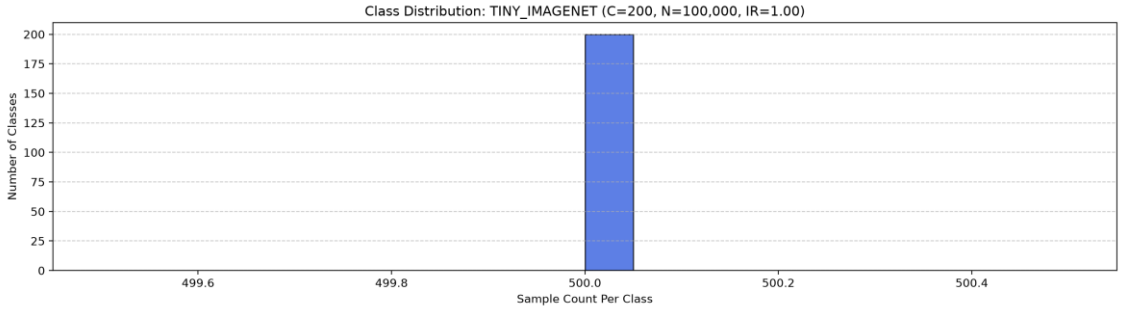


Figure 10. Tiny ImageNet-200 class distribution.

Tiny ImageNet contains 500 training images and 50 validation images per class. The class counts are balanced, but each category receives far fewer examples relative to the 200-way decision problem than CIFAR-10 receives for its ten-way problem.

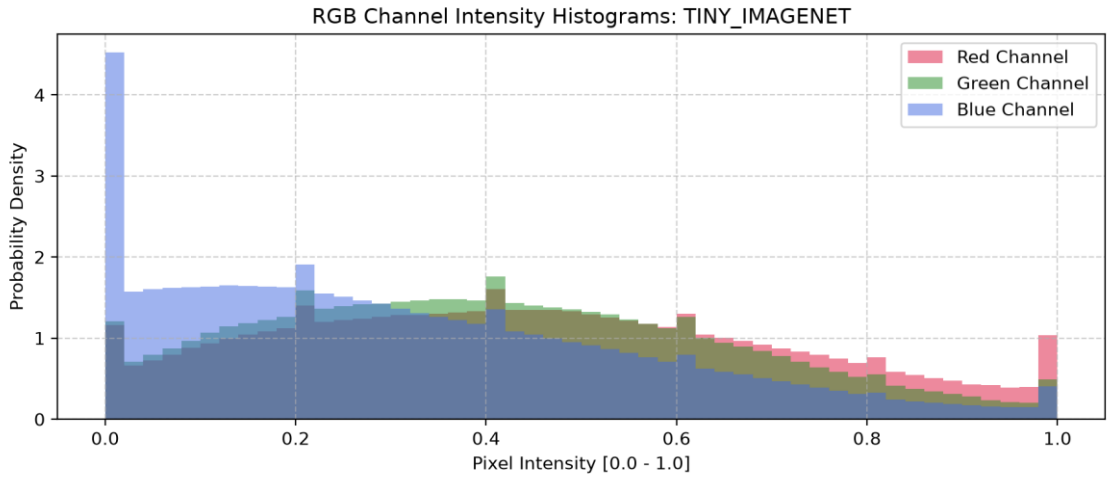


Figure 11. Tiny ImageNet channel intensity distributions.

The RGB histograms differ between CIFAR-10 and Tiny ImageNet, supporting the use of dataset-specific normalization statistics rather than one shared mean and standard deviation.

3.10 Transform pipeline output

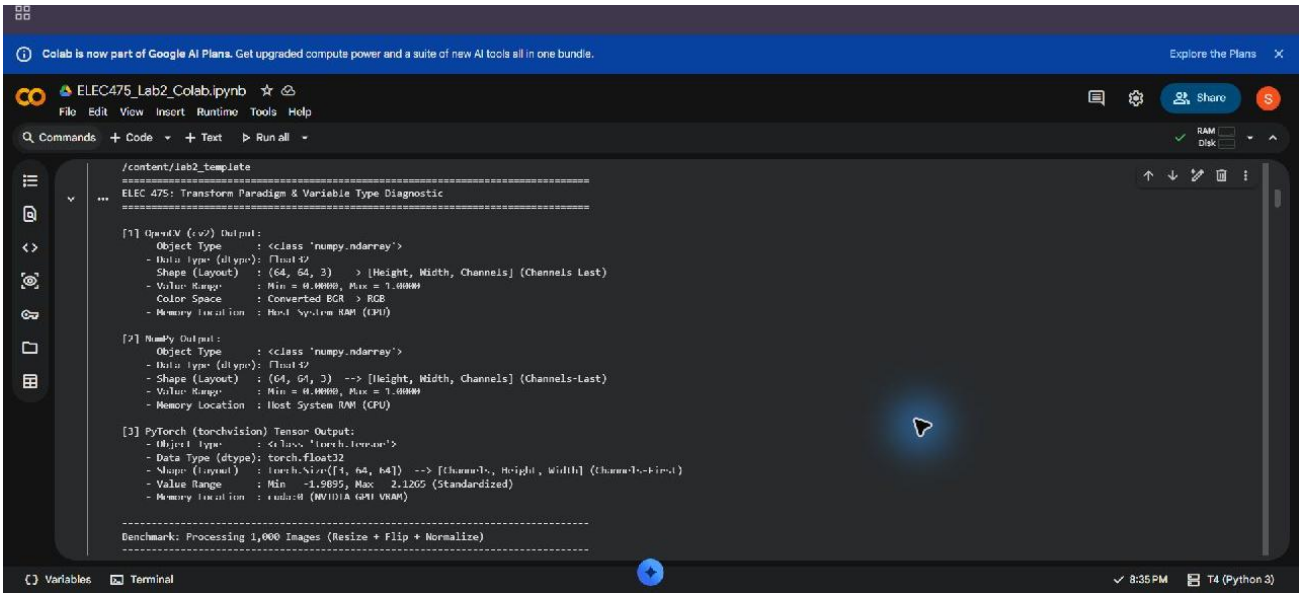


Figure 12. Fresh Google Colab output from demo_transforms.py on a Tesla T4 runtime.

Pipeline	Type and dtype	Shape	Layout	Memory	Observed range
OpenCV	numpy.ndarray float32	64 x 64 x 3	HWC	CPU RAM	0 to 1
NumPy	numpy.ndarray float32	64 x 64 x 3	HWC	CPU RAM	0 to 1
PyTorch	torch.Tensor float32	3 x 64 x 64	CHW	cuda:0	-1.9895 to 2.1265

OpenCV and NumPy kept a channels-last float32 array on the CPU with values scaled to [0, 1]. PyTorch converted the image to a channels-first tensor, standardized each channel, and moved the tensor to cuda:0. Standardization explains why the tensor range extends below zero and above one.

3.11 Model profiling output

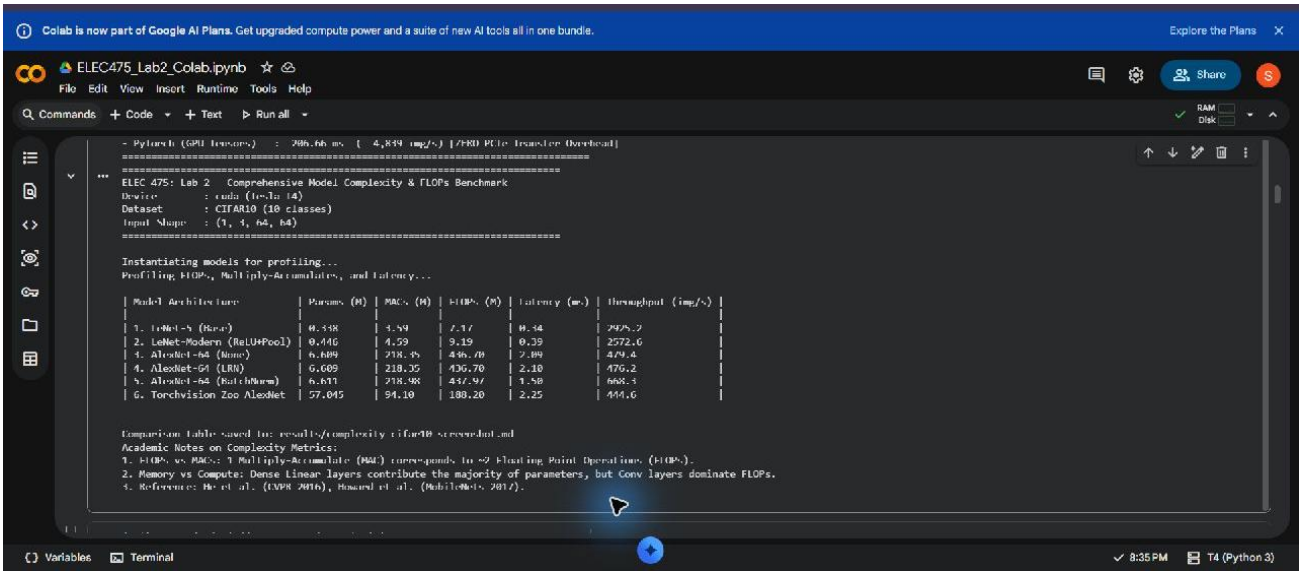


Figure 13. Fresh Google Colab terminal table from compare_models.py on a Tesla T4 runtime.

The fresh verification run reproduced the stored parameter, MAC and FLOP counts. Latency changed slightly, as expected for short GPU timing measurements affected by warm-up, runtime load and synchronization. Table 7.3a retains the recorded primary-experiment latency values so the report and saved results remain internally consistent.

4 Technical Analysis and Discussion

4.1 LeNet spatial dimensions and convergence

The dense input difference follows directly from the output-size equation. Base LeNet removes four pixels in each 5 x 5 valid convolution and halves the map in each 2 x 2 pool, ending at 16 x 13 x 13 = 2,704. Modernized LeNet pads both convolutions and uses 3 x 3 stride-2 pooling, ending at 16 x 15 x 15 = 3,600. ReLU also converges faster than Tanh because Tanh saturates at large positive or negative inputs, where its derivative approaches zero. ReLU keeps a derivative of one for positive inputs and avoids positive-side saturation. Together with overlapping max pooling, these changes increased five-epoch validation accuracy from 54.69% to 66.36%.

4.2 Normalization performance improvement

Yes. BatchNorm improved validation accuracy and macro F1 over both alternatives. It reached 78.23% accuracy and 78.20% macro F1, compared with 72.83% and 72.78% without normalization, and 73.46% and 73.46% with LRN. BatchNorm first exceeded 70% validation accuracy at epoch 4, whereas the other variants required about seven epochs.

For a channel, BatchNorm computes the mini-batch mean and variance, standardizes x to x-hat = (x - mean) / sqrt(variance + epsilon), and then learns y = gamma x-hat + beta. Keeping activation scales more consistent improves conditioning and reduces sensitivity to parameter updates, which allows gradients to propagate more effectively. This is commonly described as reducing internal covariate shift. A more cautious interpretation is that BatchNorm also reparameterizes the network so the effective optimization landscape is smoother and less sensitive to scale. LRN instead creates competition across neighboring channels: it does not center and standardize every channel's mini-batch distribution.

4.3 Normalization overhead and stability

Yes, normalization measurably changed runtime. Mean epoch time increased from 23.72 s with no normalization to 25.94 s with LRN and 24.39 s with BatchNorm. The profiler reported 436.70 MFLOPs for no normalization and LRN, and 437.97 MFLOPs for BatchNorm. The LRN operator was not fully represented in the static FLOP counter, so its unchanged nominal FLOP count should not be interpreted as zero cost: the wall-clock increase shows its actual overhead. BatchNorm added only 0.67 s per epoch and 1.27 MFLOPs while improving accuracy by 5.40 points, giving a much better benefit-to-cost ratio than LRN's 2.22 s overhead and 0.63-point gain.

All three training-loss curves decreased. By a simple epoch-to-epoch smoothness check, the unnormalized validation curve was the most monotonic, while BatchNorm had more short-horizon validation oscillation. BatchNorm nevertheless showed the best practical gradient progression because it reached useful accuracy earlier, obtained the lowest validation loss, and produced the best checkpoint. Therefore, stability here should be described as faster and more reliable optimization rather than as the visually smoothest validation curve.

4.4 Generalization gap reduction

Yes. Without augmentation, final training accuracy was 90.34% and validation accuracy was 82.14%, giving Delta = 90.34 - 82.14 = 8.20 percentage points. With augmentation, final training accuracy was 84.38% and validation accuracy was 79.73%, giving Delta = 84.38 - 79.73 = 4.65 points. The gap contracted by about 3.56 points after rounding. Random crops, horizontal flips and color jitter reduce memorization by requiring features to remain useful under label-preserving changes in position, orientation and appearance.

4.5 Validation metric improvement from augmentation

No, not within the 15-epoch schedule. The best augmented validation accuracy and macro F1 were 81.26% and 81.17%, while the unaugmented run reached 82.42% and 82.44%. The augmented training curve continued to improve while maintaining lower training accuracy, which suggests that the network had not fully assimilated the harder stochastic distribution. The transformations were moderate and semantically valid, so underfitting from insufficient training time is more plausible than destructive augmentation. A longer schedule or learning-rate decay would be the next controlled test.

4.6 Tiny ImageNet above random chance

Yes. The required AlexNet-BatchNorm run achieved 28.91% validation accuracy, far above the 0.50% random baseline. Its 26.94% macro F1 was lower than accuracy, indicating uneven class-level performance, but it still confirmed that the network learned discriminative features rather than guessing.

4.7 Cross-dataset comparison and overfitting trajectory

CIFAR-10 performance was much higher than Tiny ImageNet performance: 81.26% versus 28.91% accuracy and 81.17% versus 26.94% macro F1 for the required augmented comparison. Tiny ImageNet contains 200 fine-grained categories instead of ten, includes more varied backgrounds and viewpoints, and provides fewer training examples per class relative to the difficulty of the decision boundary. Its 64 x 64 resolution also compresses small category-specific details.

The Tiny ImageNet validation loss did not diverge or permanently plateau earlier than CIFAR-10. It had two temporary increases but reached its lowest value at epoch 10, so the model was still learning when the schedule ended. The remaining training-validation separation indicates a difficult generalization problem, but the curve supports additional training rather than an early-stop conclusion.

4.8 Model capacity and architecture fit

The manual AlexNet-BatchNorm used 6.611 million parameters and reached 78.23% in the normalization study. The Zoo AlexNet used 57.045 million parameters but reached 66.74%. The Zoo model was about 8.6 times larger, with most of its capacity in the 9,216-to-4,096 classifier projection. The smaller AlexNet-64 was better matched to 64 x 64 inputs. This result shows that parameter count alone does not determine performance: spatial design, optimization and dataset fit matter more than simply enlarging the classifier.

4.9 Overall findings

The experiments support the expected architectural progression. ReLU and overlapping max pooling improved LeNet convergence. BatchNorm gave the largest normalization gain with modest overhead. Data augmentation reduced overfitting but required more optimization time to recover the strongest short-horizon validation score. Tiny ImageNet remained much harder than CIFAR-10 but was learned far above random chance. Finally, the task-specific AlexNet-64 outperformed the much larger Zoo baseline, emphasizing the value of matching model geometry to the input scale.